

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

СЫКТЫВКАРСКИЙ ЛЕСНОЙ ИНСТИТУТ (ФИЛИАЛ) ФЕДЕРАЛЬНОГО
ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО ОБРАЗОВАТЕЛЬНОГО
УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЛЕСОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМЕНИ С. М. КИРОВА» (СЛИ)

ТРАНСПОРТНО-ТЕХНОЛОГИЧЕСКИЙ ФАКУЛЬТЕТ

КАФЕДРА ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

**ОТЧЁТ ПО ТЕХНОЛОГИЧЕСКОЙ (ПРОЕКТНО-ТЕХНОЛОГИЧЕСКОЙ)
ПРАКТИКЕ. WEB-ТЕХНОЛОГИИ**

на тему: Разработка CMS для парикмахерских и барбершопов «Сайтама»
Место прохождения практики: СЛИ
Период прохождения: 22.06.26 – 18.07.26

Выполнил: студент 3 курса
очной формы обучения

направления подготовки
бакалавриата «Информационные
системы и технологии»

_____ Решетняк В. А.
(подпись, дата)

Руководитель:
К. Э. Н., доцент

_____ Оганезова Н. А.
(подпись, дата)

Заведующий кафедрой
информационных систем и
технологий, к. ф.-м. н., доцент

_____ Плешев Д. А.
(подпись, дата)

Отчёт защищён на оценку _____

«_____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 Постановка задачи	8
1.1 Цель и задачи разработки	8
1.2 Автоматизируемые бизнес-процессы	9
2 Архитектура программного обеспечения	11
2.1 Общая схема	11
2.2 Архитектурные шаблоны бэкенда	12
2.3 Архитектура фронтенда	13
2.4 Требования законодательства и защита данных	15
3 Обоснование выбора средств реализации	18
3.1 Метод экспертных оценок	18
3.2 Список альтернатив	18
3.3 Итоговые средства реализации	21
3.4 Дополнительные инструменты разработки	23
4 Структура базы данных	26
4.1 Инфологическая модель	26
4.2 Даталогическая модель	29
4.3 Индексы и оптимизация выборок	32
5 Реализация и развёртывание приложения	37
5.1 Общие сведения	37
5.2 Создание записи (бронирование)	37
5.3 Первый запуск системы (мастер /setup)	37
5.4 Подключение аналитики	40
5.5 Размещение в сети Интернет	40
ЗАКЛЮЧЕНИЕ	41
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	42
ПРИЛОЖЕНИЕ А. Диаграммы последовательностей	44

СПИСОК СОКРАЩЕНИЙ

API – Application Programming Interface – программный интерфейс взаимодействия компонентов системы.

CI – Continuous Integration – непрерывная автоматическая проверка (тесты, линтеры) при каждом изменении кода.

CMS – Content Management System – система управления содержанием сайта.

CRM – Customer Relationship Management – управление взаимоотношениями с клиентами.

CSP – Content Security Policy – политика безопасности контента браузера.

DTO – Data Transfer Object – объект переноса данных на границе API, отделённый от внутреннего представления в БД.

HTTP – HyperText Transfer Protocol – протокол передачи данных прикладного уровня; **HTTPS** – тот же протокол поверх зашифрованного канала TLS.

JSON – JavaScript Object Notation – текстовый формат обмена данными.

JWT – JSON Web Token – формат токена авторизации, используемый для access-токена сессии.

KPI – Key Performance Indicator – ключевой показатель эффективности.

MVCC – Multiversion Concurrency Control – многоверсионное управление конкурентным доступом в СУБД.

OAuth – открытый протокол делегированной авторизации через внешнего поставщика учётных записей.

ORM – Object-Relational Mapping – отображение объектов языка программирования на строки реляционной таблицы.

REST – Representational State Transfer – архитектурный стиль построения API поверх HTTP.

SEO – Search Engine Optimization – оптимизация страниц под поисковые системы.

SPA – Single Page Application – одностраничное веб-приложение

(фронтенд).

SQL – Structured Query Language – язык запросов к реляционной СУБД.

SSR – Server-Side Rendering – формирование страницы на стороне сервера.

TLS – Transport Layer Security – протокол шифрования канала связи (HTTPS).

UI – User Interface – пользовательский интерфейс.

VK ID – сервис единой учётной записи «ВКонтакте», используемый как внешний поставщик авторизации.

БД – база данных.

СУБД – система управления базами данных (в проекте – PostgreSQL 16).

ТЗ – техническое задание.

ФТ – функциональное требование (карточки ФТ-001–ФТ-016 в техническом задании).

СПИСОК ТЕРМИНОВ

Мастер – сотрудник точки, оказывающий услуги клиентам; в системе – роль пользователя.

Слот – свободный интервал в расписании мастера, доступный для записи клиента.

Точка сети – отдельная парикмахерская или барбершоп в составе сети одного владельца – собственный адрес, часы работы, мастера и записи.

ВВЕДЕНИЕ

Предприятия сферы парикмахерских услуг и барберинга всё чаще переходят от бумажных журналов и разрозненных таблиц к специализированным программным системам. Такие решения обеспечивают онлайн-запись клиентов, ведение расписания мастеров и автоматизацию отчётности. Отсутствие единого инструмента порождает типовые для отрасли проблемы: возникновение накладок в расписании (двойных записей), неявки клиентов из-за отсутствия напоминаний, сложности с дифференцированным учётом цен мастеров, а также отсутствие у руководства оперативной аналитики по выручке и загрузке заведений. Настоящий документ посвящён веб-приложению «Сайтама», автоматизирующему данные бизнес-процессы для отдельных салонов и сетевых барбершопов.

В работе формулируется решаемая задача, описываются автоматизируемые процессы, архитектура приложения, применённые паттерны проектирования и требования нормативно-правовой базы. Обосновывается выбор технологического стека (FastAPI [1], SQLAlchemy [2], PostgreSQL [3]), приводится схема базы данных, детально рассматривается логика ключевых пользовательских сценариев, интеграция аналитических модулей и развёртывание сервиса в сети Интернет.

Комплект проектной документации включает три сопутствующих документа:

- 1) Техническое задание, оформленное в соответствии с ГОСТ 34.602-2020 [4] и ГОСТ 19.201-78 [5], содержащее постановку задачи и перечень функциональных требований (ФТ-001–ФТ-016);
- 2) Руководство пользователя, описывающее пошаговые сценарии работы для четырёх ролей: клиента, мастера, администратора филиала и владельца сети;
- 3) Руководство администратора, регламентирующее процедуры развёртывания, эксплуатации и резервного копирования серверной инфраструктуры.

Практическая значимость результатов заключается в создании готовой к практическому применению программной платформы, устраняющей издержки ручного учёта и человеческого фактора. Архитектура системы

изначально поддерживает мультифилиальность, что позволяет развёртывать её как в единичных парикмахерских, так и в разветвлённых сетях без внесения изменений в исходный код.

1 Постановка задачи

1.1 Цель и задачи разработки

Большинство предприятий малого бизнеса в сфере парикмахерских услуг и барберинга ведут запись клиентов и операционный учёт вручную – посредством телефонных звонков, бумажных журналов и разрозненных электронных таблиц. Подобный подход порождает типовые для отрасли проблемы: возникновение накладок в расписании (двойных бронирований), высокий процент неявок клиентов (no-show) из-за отсутствия автоматических напоминаний, сложности с дифференцированным расчётом стоимости услуг по мастерам, а также невозможность оперативного получения сводной аналитики по выручке и загрузке заведений без трудоёмкого ручного подсчёта.

Основной целью работы является проектирование и разработка веб-приложения «Сайтама», автоматизирующего полный цикл операционной деятельности парикмахерской или барбершопа: организацию самостоятельной онлайн-записи клиентов без участия администратора, ведение каталога услуг с индивидуальным ценообразованием, отправку автоматических напоминаний о предстоящих визитах, а также генерацию отчётности по ключевым бизнес-показателям. Архитектура разрабатываемой системы изначально предусматривает поддержку мультифилиальности с разграничением прав доступа между администратором конкретного филиала и владельцем всей сети заведений.

Достижение поставленной цели оценивается следующими измеримыми показателями (детализированными в подразделе 2.2 технического задания):

- сокращение среднего времени оформления записи со стороны клиента до 2 минут;
- полное исключение вероятности двойных записей программно-техническими средствами за счёт ограничений целостности СУБД, а не административных регламентов;
- снижение доли неявок клиентов за счёт внедрения автоматических уведомлений;

- реализация гибкой настройки прайс-листа с учётом квалификации конкретного мастера;
- консолидация данных в едином реляционном хранилище и отказ от бумажного документооборота;
- обеспечение возможности самостоятельного формирования отчётов по выручке и загруженности заведений конечными пользователями без привлечения технических специалистов.

Границы проекта строго зафиксированы: за рамками разработки оставлены модули интернет-эквайринга (расчёты производятся очно на месте оказания услуг согласно подразделу 3.2 технического задания), ведение складского и бухгалтерского учёта, а также расширенные маркетинговые инструменты CRM-системы (сегментация клиентской базы, массовые рассылки и программы лояльности).

1.2 Автоматизируемые бизнес-процессы

Система автоматизирует следующие шесть ключевых бизнес-процессов, выделенных при обследовании предметной области (техническое задание, подраздел 2.1):

- 1) **Онлайн-запись клиентов (Зап-1)** – выбор филиала, услуги, мастера и свободного временного слота с последующим подтверждением записи без участия администратора. Для новых клиентов процедура бронирования предусматривает автоматическое создание учётной записи (раздел 5).
- 2) **Управление каталогом услуг и ценообразованием (Усл-2)** – ведение номенклатуры услуг с возможностью гибкой настройки стоимости для каждой связки «мастер – услуга».
- 3) **Оповещение и напоминания о визитах (Увед-3)** – автоматическая отправка уведомлений клиентам за 24 и за 2 часа до назначенного времени приёма.
- 4) **Формирование аналитической отчётности (Отчёт-4)** – расчёт показателей выручки, загруженности мастеров и рейтинга востребованности услуг за произвольный период в пользовательском интерфейсе без необходимости прямого обращения к базе данных (5.4).

5) **Управление контентом витрины (Конт-5)** – администрирование информационной составляющей публичной части сервиса (главная страница, сведения о мастерах и их портфолио).

6) **Управление филиальной сетью (Точ-6)** – создание, редактирование и вывод из эксплуатации точек обслуживания с настройкой их адресов, контактных данных, графиков работы и разграничением прав доступа между локальными администраторами и владельцем сети (руководство пользователя, подраздел 4.4).

2 Архитектура программного обеспечения

2.1 Общая схема

Разрабатываемая система представляет собой клиент-серверное веб-приложение, состоящее из одностраничного клиентского интерфейса (SPA) на базе фреймворка Vue 3 [6] и серверной части с REST API на FastAPI [1], взаимодействующей с централизованной реляционной СУБД PostgreSQL [3].

Слоистая архитектура бэкенда и схема его взаимодействия с клиентским приложением представлены в соответствии с рисунком 1. Топология развёртывания компонентов системы (контейнеризация на базе Docker, конфигурация обратного прокси-сервера и настройка протокола TLS) детально приведена в руководстве администратора (раздел 1).

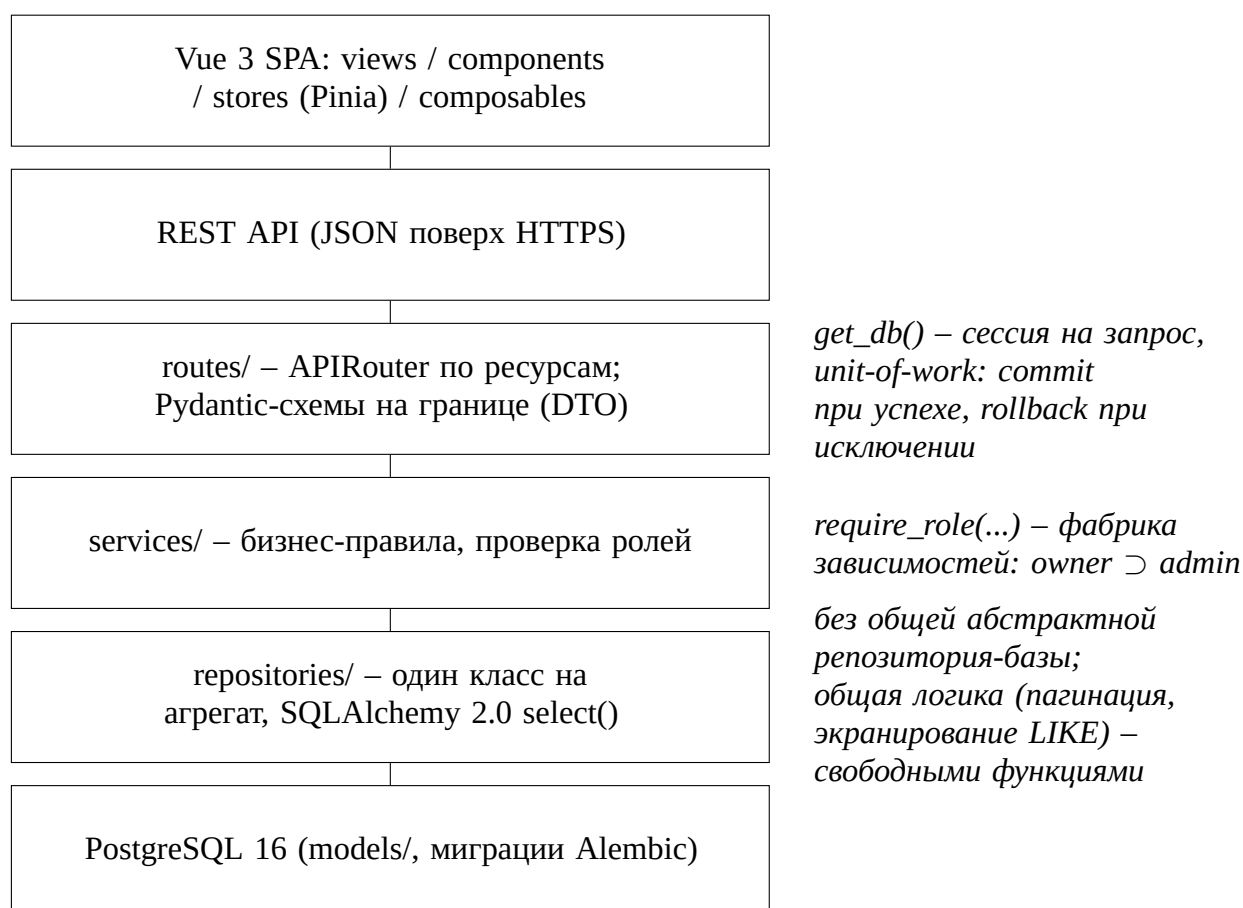


Рисунок 1 – Слоистая архитектура: от фронтенда до базы данных

2.2 Архитектурные шаблоны бэкенда

Серверная часть приложения (backend/app/) реализована на основе строгой однонаправленной слоистой архитектуры, включающей уровни маршрутизации (routes), бизнес-логики (services), доступа к данным (repositories) и объектно-реляционных моделей (models).

Вспомогательный пакет schemas/, содержащий Pydantic-схемы [7], применяется исключительно на границе обработки входящих и исходящих HTTP-запросов. Сервисный слой выполняет валидацию и сериализацию данных через вызов `XResponse.model_validate(orm_obj)`, что исключает прямую передачу внутренних ORM-сущностей во внешнее API.

В архитектуре серверной части применены следующие паттерны и подходы к проектированию:

- **Объект переноса данных (Data Transfer Object, DTO)** – разделение контрактов API (schemas/) и структур хранения базы данных (models/) обеспечивает независимость публичного интерфейса от изменений внутренней схемы данных;

- **Репозиторий (Repository)** – инкапсуляция логики взаимодействия с СУБД через SQLAlchemy [2] в специализированных классах (UserRepository, AppointmentRepository, SalonRepository). Слой бизнес-логики изолирован от формирования прямых SQL-запросов;

- **Внедрение зависимостей (Dependency Injection) и единица работы (Unit of Work)** – передача зависимостей осуществляется через встроенный механизм Depends фреймворка FastAPI. Генератор сессий `get_db()` реализует паттерн Unit of Work: в рамках одного HTTP-запроса выполняется единая транзакция с автоматической фиксацией изменений (`commit`) при успешном ответе, откатом (`rollback`) при возникновении исключений и гарантированным закрытием сессии (`close`) в блоке `finally`;

- **Фабрика зависимостей (Dependency Factory)** – параметризованная функция контроля доступа `require_role(*roles)` генерирует функции-зависимости для аутентификации пользователей (`get_current_client`, `get_current_master`, `get_current_admin`, `get_current_owner`). Поскольку права роли `owner` включают в себя все привилегии роли `admin`, проверка `get_current_admin` предоставляет доступ обеим категориям пользователей;

- **Контроль контекста филиала (Salon Scoping)** – функция `resolve_salon_scope(current_user, salon_id)` осуществляет явное разграничение области видимости данных внутри защищённых эндпоинтов. Для роли `owner` отсутствие параметра `salon_id` означает доступ ко всей сети филиалов, тогда как для роли `admin` доступ строго ограничен закреплённым за ним филиалом (при попытке несанкционированного доступа возвращается HTTP-код 403).

2.3 Архитектура фронтенда

Клиентская часть приложения (`frontend/src/`) построена на базе фреймворка Vue 3 с применением Composition API (с повсеместным использованием синтаксиса `<script setup>` на базе JavaScript), маршрутизатора Vue Router 4 и библиотеки управления состоянием Pinia. Централизованное состояние разделено на шесть изолированных хранилищ (Setup Stores): `auth`, `salon`, `setup`, `siteContent`, `masterProfile` и `toast`.

Маршрутизация реализована в модуле `router/index.js` с применением динамического импорта (ленивой загрузки) компонентов страниц. Глобальный навигационный хук `beforeEach` выполняет ролевую модель разграничения доступа на основе метаполей маршрутов (`meta.roles`), а также перенаправляет пользователей на интерфейс мастера первоначальной настройки системы (`/setup`) до завершения базовой конфигурации сервиса (5.3).

В архитектуре фронтенда реализованы два ключевых решения, обеспечивающих безопасность и гибкость клиентской части:

- **Безопасное хранение и обновление токенов аутентификации** – токен сессии передаётся исключительно через cookie-файлы с флагом `HttpOnly`, исключая его хранение в `localStorage` или переменных JavaScript, что минимизирует риски компрометации через XSS-атаки. Обновление пары токенов реализовано через перехватчик (interceptor) библиотеки `Axios` по паттерну `Single-Flight`: при получении ответа с кодом 401 инициируется единственный запрос к эндпоинту `POST/auth/refresh`, в то время как параллельные асинхронные вызовы ожидают разрешения единого промиса (`Promise`). Полный сброс сессии и перенаправление на форму входа (`logout`) выполняются только в случае неуспешного ответа самого запроса на обновление токена.
- **Динамическая тема оформления** – корпоративная цветовая

палитра (brand, accent, ink, stone) объявлена в конфигурации Tailwind CSS через пользовательские CSS-переменные (CSS Custom Properties). Значения переменных динамически считываются из конфигурации сайта в runtime, что позволяет адаптировать визуальный стиль под бренд конкретного филиала без повторной компиляции исходного кода фронтенда.

Структуру интерфейса и его поведение при нештатных состояниях определяют следующие решения:

- **Навигация** – публичная часть сайта использует горизонтальное главное меню, тогда как оба закрытых раздела построены на боковой панели навигации (компонент `components/SidebarLink.vue`): административная панель содержит восемь разделов (статистика, пользователи, услуги, мастера, точки сети, отзывы, отчёты, настройки), личный кабинет мастера – три (записи, расписание, отзывы). Состав панели зависит от роли: пункты «Точки сети» и «Настройки» отображаются только владельцу сети, поскольку соответствующие эндпоинты закрыты для роли admin (2.2).

- **Двусторонняя валидация форм** – ограничения полей проверяются дважды. На клиенте набор функций-валидаторов (`utils/validators.js`) в связке с `composable`-функцией `composables/useFormErrors.js` подсвечивает ошибку под полем до отправки запроса; на сервере те же ограничения независимо проверяются схемами Pydantic. Клиентская проверка дублирует серверные правила и не может их ослаблять: единственным авторитетом остаётся сервер, а клиентская половина лишь заменяет ответ с кодом 422 немедленной подсветкой поля.

- **Система интерфейсных компонентов** – визуальный стиль задан не набором разрозненных классов, а библиотекой из двадцати базовых компонентов (`components/ui/`: элементы ввода, кнопки, карточки, диалоги подтверждения, индикаторы состояния загрузки, уведомления). Оформление опирается на согласованные наборы значений, объявленные в конфигурации Tailwind CSS: палитра, шкала скруглений и трёхступенчатая шкала теней. Такое разделение позволяет менять оформление централизованно, не затрагивая разметку отдельных страниц.

- **Страницы ошибочных состояний** – обращение к несуществующему маршруту отображает страницу с кодом 404 и переходом в каталог мастеров. Ответ сервера с кодом 5xx перехватывается тем же перехватчиком

Axios, что и код 401, и приводит к переходу на страницу с кодом 500, содержащую пояснение о характере сбоя и контактный телефон точки сети. Сетевые ошибки (отсутствие ответа сервера) к такому переходу намеренно не приводят: их причина, как правило, находится на стороне клиента, и переход на страницу ошибки означал бы потерю заполненной формы.

– **Автоматическое обновление статистики** – показатели административной панели обновляются без участия пользователя с интервалом 30 секунд (composables/usePolling.js). Очередной запрос планируется только после завершения предыдущего, что исключает наложение запросов при медленном ответе сервера; на неактивной вкладке браузера опрос приостанавливается и возобновляется немедленным обновлением при возврате к ней. Сбой фоновой обновления не прерывает работу с панелью: ранее полученные значения сохраняются на экране с отметкой времени их получения.

2.4 Требования законодательства и защита данных

В соответствии с техническим заданием (подраздел 1.4), система спроектирована с учётом требований Федерального закона № 152-ФЗ «О персональных данных» [8] и Федерального закона № 149-ФЗ «Об информации, информационных технологиях и о защите информации» [9]. Методология оценки достаточности реализованных мер защиты опирается на положения ГОСТ Р ИСО/МЭК 15408-3-2008 [10].

В рамках программного решения реализован следующий комплекс мер по обеспечению информационной безопасности и защите персональных данных:

– **Фиксация согласия на обработку персональных данных** – форма регистрации предусматривает обязательное подтверждение согласия пользователя посредством интерактивного элемента (чекбокса) с активной гиперссылкой на текст политики конфиденциальности. Отправка формы клиентом блокируется до момента подтверждения; аналогичный механизм валидации согласия реализован для сценария авторизации через внешний сервис VK ID.

– **Публикация правовых документов** – на клиентском маршруте /privacy размещена страница политики конфиденциальности, структуриро-

ванная в соответствии с требованиями 152-ФЗ (определены правовые основания, категории обрабатываемых данных, цели обработки, права субъектов, условия применения аналитических инструментов и контактные данные оператора).

- **Информирование об использовании cookie-файлов и сервисов аналитики** – в п. 8 политики конфиденциальности раскрыты сведения об обработке cookie-файлов и интеграции со счётчиком Яндекс.Метрики (5.4).

- **Безопасность хранения и управления данными** – аутентификационные данные не сохраняются в открытом виде: в базе данных фиксируются исключительно односторонние криптографические хеши (поле password_hash). Физическое удаление учётной записи пользователя при наличии связанных с ней записей на приём предотвращается на уровне схемы реляционной БД с помощью ограничения ссылочной целостности (ON DELETE RESTRICT). Для прекращения обслуживания учётной записи применяется механизм логической блокировки (флаг is_blocked).

- **Вход через внешнего поставщика учётных записей** – помимо пары «адрес электронной почты и пароль», поддерживается авторизация через сервис VK ID по протоколу OAuth 2.0. Применён классический серверный поток с перенаправлением браузера (GET/auth/vk/login → VK → GET/auth/vk/callback), дополненный механизмом PKCE (метод S256) и параметром state; оба значения хранятся в cookie-файлах с флагом HttpOnly, ограниченных путём маршрута обратного вызова, и защищают обмен кода на токен от подмены и от межсайтовой подделки запроса. Учётная запись, созданная таким способом, не содержит пароля вовсе: поле password_hash допускает значение NULL, а связь с внешней учётной записью хранится в поле vk_user_id. Для ролей owner и admin этот способ входа отключается, если задан код администратора: в потоке OAuth предъявить его негде, и такой вход обходил бы второй фактор.

- **Противодействие подбору учётных данных** – каждая попытка входа фиксируется в таблице login_attempt_log с указанием адреса электронной почты, IP-адреса и результата. После пяти неудачных попыток вход по данному адресу временно блокируется на 15 минут; запросы на восстановление пароля ограничены тремя в час. Журнал попыток одновременно выполняет функцию аудита: он позволяет отличить единичную ошибку

пользователя от целенаправленного перебора.

– **Второй фактор доступа к административной панели** – помимо пароля, роли owner и admin предъявляют при входе общий код администратора (параметр конфигурации ADMIN_LOGIN_TOKEN). Код проверяется после пароля и сравнивается функцией secrets.compare_digest, устойчивой к атакам по времени выполнения; неудачная проверка фиксируется в том же журнале login_attempt_log, вследствие чего подбор кода ограничен той же блокировкой, что и подбор пароля. При неверном пароле сообщение об ошибке не изменяется даже при верном коде – в противном случае различие в ответах указывало бы на успешный подбор пароля. Для ролей client и master код не запрашивается, поскольку административная панель закрыта для них ролевой моделью (2.2). Параметр является необязательным: при пустом значении вход выполняется по паролю, что позволяет разворачивать систему без дополнительной настройки.

– **Защита от межсайтовых запросов** – cookie-файлы с токенами выпускаются с атрибутом SameSite=Lax, вследствие чего браузер не передаёт их при межсайтовых запросах методами POST, PATCH и DELETE. Перечень источников, которым разрешено обращаться к программному интерфейсу, задаётся явным списком в конфигурации сервера (механизм CORS) и не допускает произвольного происхождения запроса.

3 Обоснование выбора средств реализации

3.1 Метод экспертных оценок

Для обоснованного выбора технологического стека применяется метод ранжировки с расчётом интегрального показателя полезности. Метод позволяет формализовать экспертную оценку при сравнении альтернатив по нескольким критериям с различным приоритетом; экспертом выступает единственный исполнитель проекта.

Коэффициент ценности C_{ij} для произвольного ранга критерия r_{ij} ($1 \leq r_{ij} \leq n$):

$$C_{ij} = 1 - \frac{r_{ij} - 1}{n}, \quad (3.1)$$

где C_{ij} – коэффициент ценности i -го критерия;
 r_{ij} – ранг критерия (1,0 – наиболее значимый);
 n – общее число критериев.

Нормированный вес i -го критерия b_i :

$$b_i = C_{ij} / \sum_{i=1}^n C_{ij}. \quad (3.2)$$

Интегральный показатель полезности альтернативы E :

$$E = \sum_{i=1}^n b_i q_i, \quad (3.3)$$

где E – интегральный показатель полезности альтернативы;
 b_i – нормированный вес i -го критерия;
 $q_i \in [0; 1]$ – нормированная оценка альтернативы по i -му критерию (1,00 присваивается лучшей альтернативе по данному критерию).

3.2 Список альтернатив

Рассматриваются технологии с открытой лицензией и широкой распространённостью на российском рынке разработки; учитывается доступность специалистов и отсутствие лицензионных ограничений для разработ-

чиков в РФ.

Серверная часть. Рассматриваются Python, Node.js, Java и PHP. Python обладает развитой экосистемой библиотек для работы с базами данных, построения REST API и организации фоновых задач. Node.js – популярная платформа для серверной разработки на JavaScript с активным сообществом. Java – зрелое корпоративное решение с богатой инфраструктурой, но с более высоким порогом входа и длительной первоначальной настройкой. PHP сохраняет значительную долю рынка корпоративных веб-приложений. При выборе серверной части приоритет отдан доступности специалистов и скорости разработки: нагрузка малой парикмахерской или сети из нескольких точек не является критичным фактором, поэтому производительность и масштабируемость имеют меньший вес. Критерии оценивания серверной части приведены в соответствии с таблицей 1.

Таблица 1 – Критерии и веса оценки серверной части

Обозначение	Критерий	Ранг r	C_i	b_i
q_1	Производительность	6,5	0,313	0,070
q_2	Надёжность, наличие готовых библиотек	4	0,625	0,139
q_3	Стоимость и лицензирование	3	0,750	0,167
q_4	Безопасность	2	0,875	0,194
q_5	Документация и сообщество	8	0,125	0,028
q_6	Масштабируемость	6,5	0,313	0,070
q_7	Скорость разработки	1	1,000	0,222
q_8	Простота интеграции со сторонними сервисами	5	0,500	0,111

Клиентская часть. Рассматриваются Vue 3, React, Next.js и Angular. Vue 3 и React – наиболее распространённые библиотеки для построения одностраничных приложений (SPA). Next.js – фреймворк на основе React с поддержкой серверного рендеринга (SSR), избыточного для проекта без требований к SEO публичной части. Angular – полноценный фреймворк со

встроенной маршрутизацией, управлением состоянием и внедрением зависимостей, но с более высоким порогом входа. При выборе клиентской части приоритет отдан скорости разработки UI и низкому порогу входа: система включает как публичный интерфейс записи, так и административную панель, разрабатываемые одним исполнителем. Критерии оценивания клиентской части приведены в соответствии с таблицей 2.

Таблица 2 – Критерии и веса оценки клиентской части

Обозначение	Критерий	Ранг r	C_i	b_i
q_1	Скорость разработки UI, компактность API реактивности	1	1,000	0,400
q_2	Порог входа для стороннего разработчика	2	0,750	0,300
q_3	Сложность интеграции с backend (REST API)	3	0,500	0,200
q_4	Качество документации	4	0,250	0,100

Хранение данных. Рассматриваются PostgreSQL, MariaDB и SQLite. Все три СУБД распространяются по открытым лицензиям. PostgreSQL – объектно-реляционная СУБД с развитыми механизмами обеспечения целостности данных и конкурентного доступа. MariaDB – развитие MySQL, широко применяемое в веб-разработке. SQLite – встраиваемая СУБД без отдельного серверного процесса, с существенными ограничениями в части конкурентной записи. При выборе СУБД целостность данных и конкурентный доступ – первостепенный критерий: несколько администраторов и клиентов одновременно работают с расписанием, и двойное бронирование одного временного слота должно быть исключено средствами самой СУБД (многоверсионное управление конкурентным доступом, MVCC; уровни изоляции транзакций; ограничения целостности), а не только проверкой в коде приложения. Критерии оценивания СУБД приведены в соответствии с таблицей 3.

Таблица 3 – Критерии и веса оценки СУБД

Обозначение	Критерий	Ранг r	C_i	b_i
q_1	Поддержка конкурентного доступа и изоляции транзакций	1	1,000	0,279
q_2	Ограничения целостности на уровне схемы (EXCLUDE, диапазонные типы)	2	0,833	0,233
q_3	Простота администрирования и резервного копирования	3	0,667	0,186
q_4	Поддержка со стороны российских managed-хостингов	4	0,500	0,140
q_5	Производительность при индексируемых выборках	5	0,333	0,093
q_6	Лицензия и отсутствие санкционных ограничений	6	0,167	0,047

3.3 Итоговые средства реализации

Итоговые результаты экспертной оценки представлены в соответствии с таблицей 4.

Таблица 4 – Итоговые результаты экспертной оценки

Категория	Средство	E
Серверная часть	Python (FastAPI)	0,904
Клиентская часть	Vue 3	0,930
СУБД	PostgreSQL	0,978

Python демонстрирует наивысший интегральный показатель (0,904) прежде всего за счёт лидерства по критерию скорости разработки, имеющему

наибольший вес (0,222). Фреймворк FastAPI [1] обеспечивает встроенную валидацию данных через Pydantic [7], автоматическую генерацию интерактивной документации API (OpenAPI/Swagger, /api/docs) и асинхронную обработку запросов – полноценный REST API реализуется за минимальное время без отдельного описания контракта вручную. Экосистема Python предоставляет зрелые библиотеки для всех задач проекта: SQLAlchemy 2.0 [2] (типизированный ORM), Alembic [11] (версионизируемые миграции схемы), python-jose (JWT). Весь стек распространяется под открытыми лицензиями без ограничений для российских разработчиков.

Фреймворк Vue 3 занял лидирующую позицию с интегральным показателем 0,930 благодаря максимальной оценке по критерию скорости разработки пользовательского интерфейса (весовой коэффициент 0,400). Использование Composition API в сочетании с библиотекой управления состоянием Pinia позволяет создавать реактивный интерфейс без избыточного шаблонного кода (boilerplate), характерного для связок Vuex/Redux или системы внедрения зависимостей Angular, что оптимально для масштаба разрабатываемого сервиса. Интеграция с REST API бэкенда стандартизирована, а исчерпывающая документация фреймворка минимизирует порог входа и упрощает сопровождение клиентской части.

PostgreSQL демонстрирует наивысший интегральный показатель (0,978) и уверенно лидирует по двум наиболее весомым критериям. По конкурентному доступу PostgreSQL не имеет равных среди рассмотренных альтернатив: механизм MVCC и поддержка уровней изоляции транзакций гарантируют корректность при одновременных попытках бронирования одного слота несколькими клиентами. Дополнительную защиту обеспечивает ограничение целостности EXCLUDE USING gist с диапазоным типом tstzrange: требование «0 двойных бронирований» (техническое задание, подраздел 2.2) решено на уровне схемы БД, а не проверкой в коде приложения перед записью, которая при параллельных запросах подвержена состоянию гонки. PostgreSQL распространяется под лицензией PostgreSQL License (аналог MIT) и доступен в виде managed-сервиса на отечественных облачных платформах. SQLite исключён ввиду отсутствия полноценной поддержки конкурентной записи; MariaDB уступает PostgreSQL по глубине поддержки механизмов изоляции

транзакций и диапазонных ограничений целостности.

Вспомогательные средства реализации, не участвовавшие в сравнении альтернатив ввиду отсутствия конкурирующих решений того же назначения в рамках проекта, – Tailwind CSS (единая цветовая и типографическая система без написания собственного CSS-фреймворка с нуля) и Docker Compose [12] с Caddy [13] (воспроизводимое развёртывание и автоматический TLS-сертификат Let’s Encrypt без ручной настройки nginx/certbot; подробности – руководство администратора, раздел 1).

Обоснование отказа от распределённой очереди задач (Celery / Redis). Рассылка уведомлений клиентам реализована в виде фонового асинхронного цикла (asuncio) внутри основного процесса бэкенда без привлечения внешнего брокера сообщений (техническое задание, подраздел 3.3). Архитектура системы рассчитана на эксплуатацию в рамках одного контейнера backend (руководство администратора, раздел 1). В этих условиях внедрение внешнего брокера очередей привело бы к неоправданному усложнению инфраструктуры и увеличению накладных расходов. По аналогичной причине ограничение частоты запросов (rate limiting) и защита от подбора аутентификационных данных реализованы без использования внешнего кэша Redis: лимиты запросов контролируются библиотекой slowapi в оперативной памяти процесса по IP-адресам, а фиксация попыток входа осуществляется в служебной таблице login_attempt_log базы данных.

3.4 Дополнительные инструменты разработки

Вспомогательные инструменты линтинга, тестирования и непрерывной интеграции, использованные при разработке, перечислены в соответствии с таблицей 5.

Таблица 5 – Инструменты разработки, тестирования и контроля качества

Инструмент	Назначение
ruff	Линтер Python (E,F,B,UP,SIM,C4).
ESLint 9 + eslint-plugin-vue	Линтер JS/Vue (flat config, vue/flat/essential).

Продолжение таблицы 5

Инструмент	Назначение
Pyright	Проверка типов Python в режиме basic (локально).
pytest (unit + integration)	Модульные тесты (backend/tests) и отдельно помеченные интеграционные тесты (backend/tests/integration) на настоящем Postgres через httpx.AsyncClient – покрывают ограничение EXCLUDE, уникальность slug точки, область видимости по точкам сети, полный сценарий записи «от начала до конца».
Vitest + @vue/test-utils	Модульные тесты фронтенда (хранилища, composables).
Playwright + @axe-core/playwright	End-to-end тесты в браузере (Chromium) и автоматическая регрессия по доступности (accessibility).
GitHub Actions (CI)	Параллельные job'ы на каждый push/PR в main: тесты и линт бэкенда, интеграционные тесты бэкенда на реальном Postgres, сборка и линт фронтенда, unit- и e2e-тесты фронтенда, синтаксическая проверка docker-compose.yml.
pip-audit / npm audit	Проверка зависимостей на известные уязвимости как отдельные шаги CI.
Dependabot	Еженедельные автоматические PR на обновление зависимостей (pip, npm, Docker-образы обоих Dockerfile, GitHub Actions).

Работа велась в системе контроля версий Git с размещением репозитория на сервисе GitHub. Принята двухветочная модель: ветвь main содержит состояние, соответствующее развёрнутой версии, разработка ведётся в ветви develop с последующим слиянием. К моменту подготовки настоящего

документа история насчитывает более 150 фиксаций (commit); сообщение каждой из них оформлено по соглашению Conventional Commits (префиксы feat, fix, docs, chore) и содержит пояснение причины изменения, а не только его состава.

4 Структура базы данных

4.1 Инфологическая модель

Модель разделена на две схемы для читаемости: в соответствии с рисунком 2 – сущности бизнес-домена (точки, пользователи, мастера, услуги, записи, отзывы); в соответствии с рисунком 3 – сущности, обеспечивающие безопасность входа (сессии, токены, попытки входа), и настройки сайта.

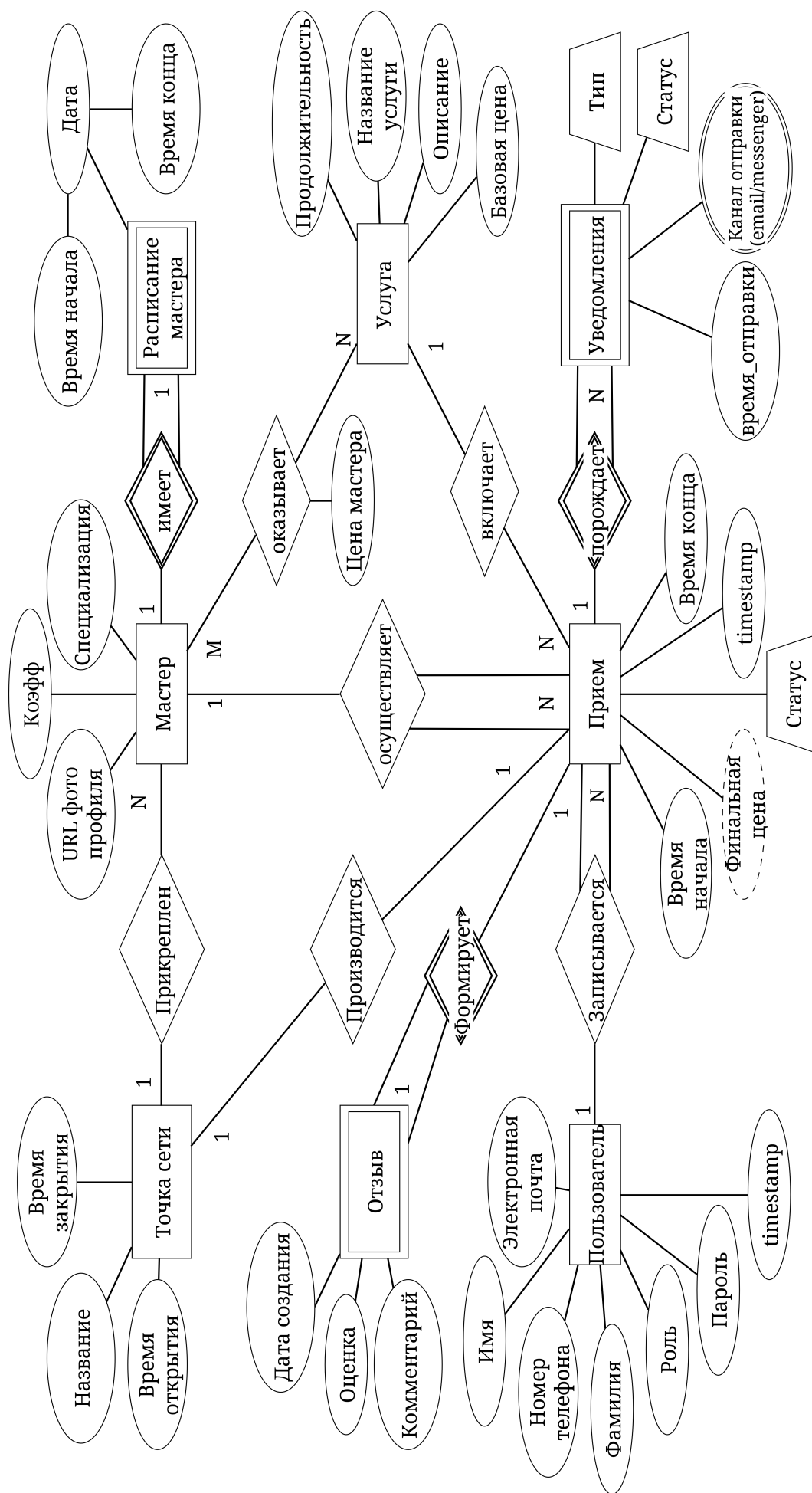


Рисунок 2 – Инфологическая модель – бизнес-домен (актуальная схема БД)

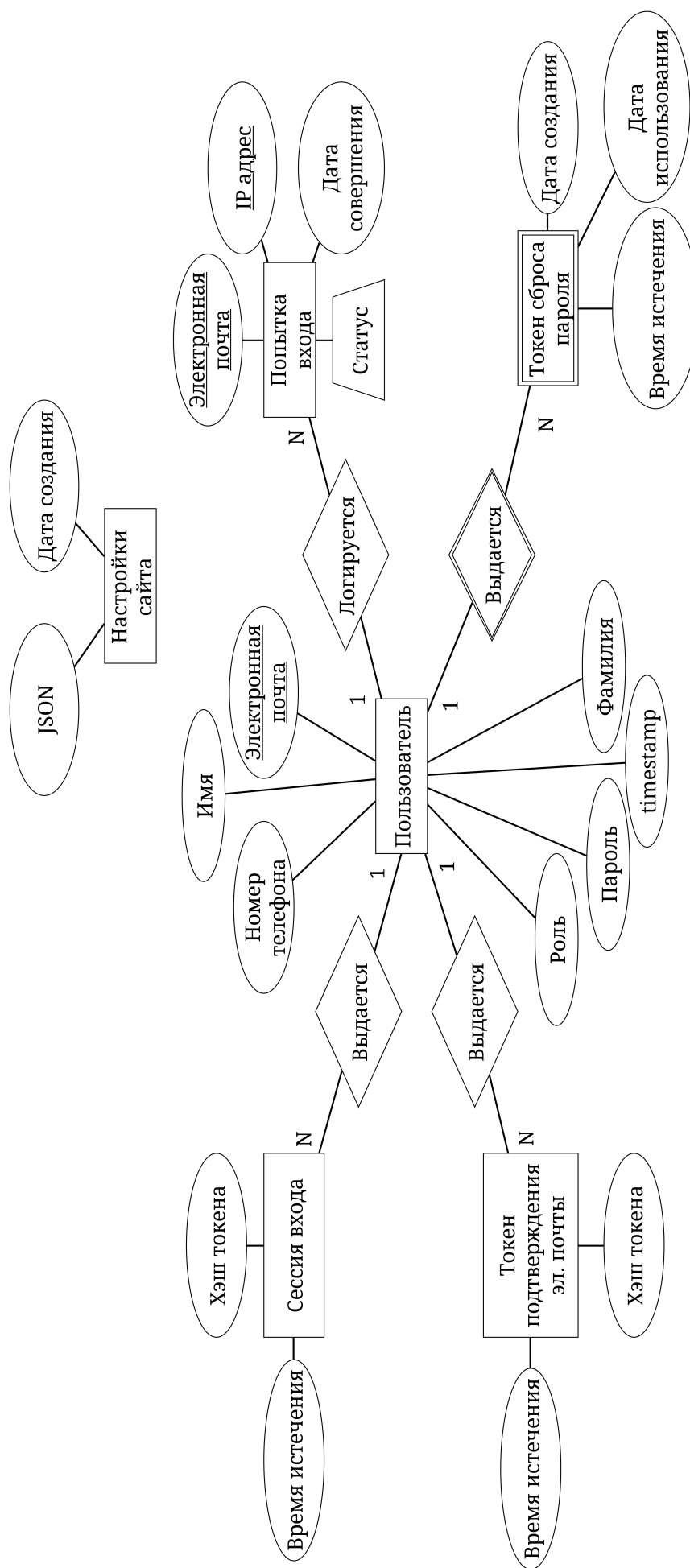


Рисунок 3 – Инфологическая модель – аутентификация и настройки сайта

4.2 Даталогическая модель

Именованние объектов схемы подчинено принятому в проекте стандарту: идентификаторы записаны в нотации snake_case строчными латинскими буквами, имя таблицы задаётся в единственном числе и отражает тип сущности, а зарезервированные слова SQL не используются – отсюда user_account вместо user. Связующая таблица именуется по двум связываемым сущностям в алфавитном порядке (master_service), таблица журнала – с суффиксом _log (login_attempt_log). Поля внешних ключей следуют шаблону «имя целевой таблицы + _id»; при нескольких связях с одной и той же таблицей добавляется префикс роли (client_id и master_id в таблице appointment – обе ссылаются на user_account).

Фактическая схема БД насчитывает 13 таблиц (15 миграций Alembic [11] в директории backend/alembic/versions/):

- 1) salon – точки сети (адрес, часы работы, slug);
- 2) user_account – учётные записи всех ролей (client/master/admin/owner);
- 3) master – профиль мастера (специализация, коэффициент к цене);
- 4) schedule – рабочее расписание мастера по дням недели;
- 5) service – каталог услуг (цена, длительность);
- 6) master_service – прайс услуги, индивидуальный по мастеру;
- 7) appointment – записи на приём (EXCLUDE USING gist – защита от двойного бронирования на уровне СУБД);
- 8) review – отзывы клиентов о завершённых записях;
- 9) user_session – серверное хранилище refresh-сессий;
- 10) password_reset_token – одноразовые токены сброса пароля;
- 11) email_verification_token – одноразовые токены подтверждения email;
- 12) login_attempt_log – аудит попыток входа;
- 13) site_setting – редактируемый контент сайта (единственная строка, JSONB).

Схема приведена к третьей нормальной форме. Помимо базовых типов применяются расширенные типы данных СУБД PostgreSQL. Перечис-

лимые типы (ENUM) объявлены на уровне схемы, а не эмулируются строковыми полями с проверочным ограничением: тип `user_role` задаёт четыре роли (`client`, `master`, `admin`, `owner`), тип `appointment_status` – четыре состояния записи (`pending`, `confirmed`, `cancelled`, `done`), образующие машину состояний записи на приём. Тип JSONB использован для таблицы `site_setting`, хранящей редактируемый администратором контент сайта переменной структуры.

Удаление данных реализовано в двух формах, различающихся по назначению. Для учётных записей, профилей мастеров и услуг применяется логическое (мягкое) удаление: примесный класс `SoftDeleteMixin` добавляет таблице поле `deleted_at`, а сама строка сохраняется, вследствие чего связанные с ней записи на приём и отзывы остаются доступными для отчётности. Уникальность значений при этом обеспечивают частичные индексы с условием `WHERE deleted_at IS NULL` (4.3), что позволяет повторно зарегистрировать адрес электронной почты, ранее принадлежавший удалённой учётной записи. Для точек сети мягкое удаление намеренно не применяется: точка либо закрывается флагом `is_active`, либо не удаляется вовсе – физическое удаление предотвращается ограничением ссылочной целостности `ON DELETE RESTRICT` со стороны мастеров и записей.

Даталогическая модель данных приведена в соответствии с рисунком 4.

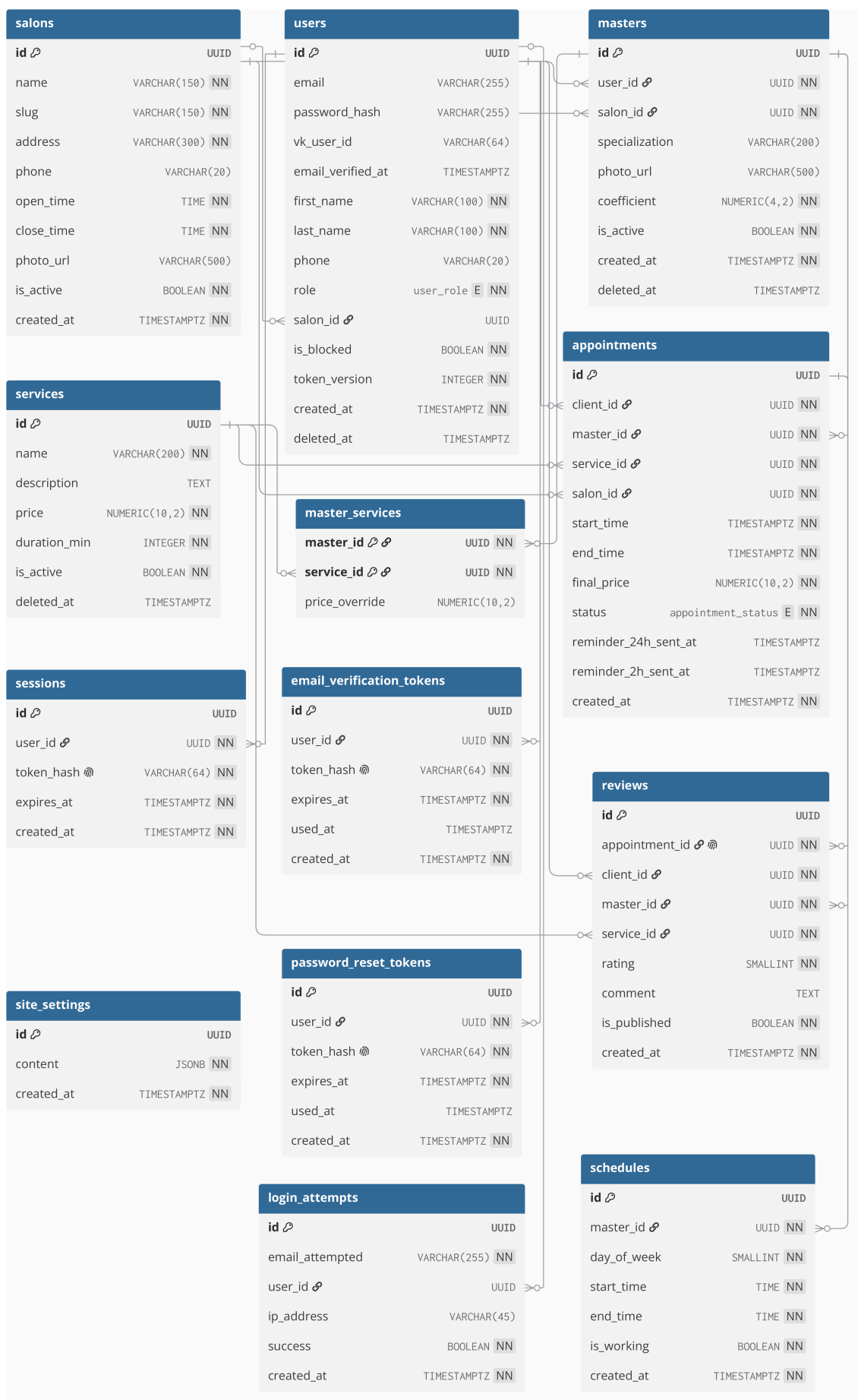


Рисунок 4 – Даталогическая модель – физическая схема БД

4.3 Индексы и оптимизация выборки

Помимо индексов, создаваемых СУБД автоматически для первичных ключей и ограничений уникальности, схема содержит 21 явно объявленный индекс. Индексы создаются миграциями Alembic вместе с таблицами, к которым относятся, и объявлены в моделях предметной области (backend/app/models/), вследствие чего схема, описанная в коде, и схема, существующая в базе данных, не расходятся между собой. Имена индексов подчинены тому же стандарту, что и имена таблиц (4.2): обычный индекс получает префикс idx_, уникальный – uniq_, далее следуют имя таблицы и перечень полей в порядке их объявления. Назначение основных индексов приведено в соответствии с таблицей 6.

Таблица 6 – Индексы схемы данных и их назначение

Индекс	Назначение
uniq_user_account_email	Частичный уникальный индекс по адресу электронной почты с условием deleted_at IS NULL. Обеспечивает уникальность адреса среди действующих учётных записей и одновременно обслуживает поиск пользователя при входе. Аналогично построены индексы по номеру телефона и идентификатору VK ID.
idx_user_account_role	Выборка пользователей по роли – список клиентов и назначение прав в административной панели.
idx_user_account_salon_id	Частичный индекс по точке сети с условием salon_id IS NOT NULL: разграничение области видимости данных для роли admin (2.2).

Продолжение таблицы 6

Индекс	Назначение
idx_appointment_start_time_end_time	Составной индекс по границам интервала (start_time, end_time) – расчёт свободных интервалов мастера, наиболее частая выборка в системе.
idx_appointment_master_id	Записи конкретного мастера – личный кабинет и расписание.
idx_appointment_client_id	Записи конкретного клиента – история посещений в личном кабинете.
idx_appointment_status	Отбор записей по состоянию: рассылка напоминаний и отчёты по завершённым записям.
idx_appointment_salon_id	Записи в границах одной точки сети – статистика и отчёты филиала.
idx_master_salon_id	Мастера, закреплённые за точкой сети.
idx_service_name	Поиск услуги по названию в каталоге.
idx_review_master_id	Отзывы о мастере и расчёт его средней оценки.
idx_review_service_id	Отзывы об услуге.
idx_login_attempt_log_email_attempted_created_at	Составной индекс (email_attempted, created_at): подсчёт неудачных попыток входа за последние 15 минут (2.4). Выполняется при каждой попытке входа, вследствие чего отсутствие индекса замедляло бы авторизацию по мере роста журнала.
idx_user_session_user_id	Активные сессии пользователя: обновление токена и отзыв всех сессий администратором.
idx_password_reset_token_user_id	Одноразовые токены сброса пароля по владельцу.

Продолжение таблицы 6

Индекс	Назначение
idx_email_verification_token_user_id	Одноразовые токены подтверждения адреса электронной почты по владельцу.
uniq_salon_slug	Уникальность и поиск точки сети по человекочитаемому идентификатору в адресе страницы.
idx_master_service_service_id	Обратный обход связи «многие ко многим»: перечень мастеров, оказывающих заданную услугу.

Отдельно следует отметить ограничение EXCLUDE USING gist на таблице appointment (4.2), которое требует расширения btree_gist и опирается на собственный индекс типа GiST. Это ограничение выполняет двойную функцию: предотвращает пересечение интервалов времени у одного мастера на уровне СУБД и ускоряет проверку пересечений, поскольку поиск конфликтующего интервала выполняется по индексу, а не последовательным просмотром таблицы.

Корректность применения индексов проверяется командой EXPLAIN ANALYZE на плане запроса расчёта занятых интервалов мастера – наиболее частой выборки системы:

```
EXPLAIN ANALYZE
SELECT start_time, end_time FROM appointment
WHERE master_id = '...' AND status <> 'cancelled'
AND start_time >= '2026-09-08' AND start_time < '2026-09-09';
```

Замер выполнен на базе, наполненной 40 000 записей на приём за два года по восьми мастерам; перед замером собрана статистика (ANALYZE). План выполнения приведён в соответствии с рисунком 5.

QUERY PLAN

```
Bitmap Heap Scan on appointment (cost=64.08..147.85 rows=18
  width=16) (actual time=0.119..0.123 rows=24.00 loops=1)
  Recheck Cond: ((start_time >= '2025-06-01 00:00:00+03'::timestamp
    with time zone) AND (start_time < '2025-06-02
    00:00:00+03'::timestamp with time zone) AND (master_id =
    '31785464-6376-4956-9b25-52b88c074d84'::uuid))
  Filter: (status <> 'cancelled'::appointment_status)
  Heap Blocks: exact=4
  Buffers: shared hit=11
-> BitmapAnd (cost=64.08..64.08 rows=24 width=0) (actual
  time=0.112..0.112 rows=0.00 loops=1)
  Buffers: shared hit=7
  -> Bitmap Index Scan on
  idx_appointment_start_time_end_time (cost=0.00..6.21
  rows=192 width=0) (actual time=0.013..0.014 rows=192.00
  loops=1)
    Index Cond: ((start_time >= '2025-06-01
    00:00:00+03'::timestamp with time zone) AND
    (start_time < '2025-06-02 00:00:00+03'::timestamp
    with time zone))
    Index Searches: 1
    Buffers: shared hit=2
  -> Bitmap Index Scan on idx_appointment_master_id
  (cost=0.00..57.61 rows=4976 width=0) (actual
  time=0.097..0.097 rows=5000.00 loops=1)
    Index Cond: (master_id =
    '31785464-6376-4956-9b25-52b88c074d84'::uuid)
    Index Searches: 1
    Buffers: shared hit=5

Planning:
  Buffers: shared hit=192
Planning Time: 0.302 ms
Execution Time: 0.149 ms
(19 rows)
```

Рисунок 5 – План выполнения запроса расчёта занятых интервалов

Планировщик выбрал не последовательный просмотр таблицы, а пересечение двух битовых карт (BitmapAnd), построенных по обоим индексам: idx_appointment_start_time_end_time отбирает записи в границах суток, idx_appointment_master_id – записи конкретного мастера. Из 40 000 строк прочитано четыре страницы данных (Heap Blocks: exact=4), время выполнения – 0,149 мс при времени планирования 0,302 мс, то есть построение плана обходится дороже самого чтения. Условие по состоянию записи применяется

уже после выборки (Filter), и отдельный индекс по status для этого запроса не нужен: он существует ради других выборок (таблица 6).

5 Реализация и развёртывание приложения

5.1 Общие сведения

Полный перечень операций по каждой из четырёх ролей – руководство пользователя, подразделы 4.1–4.4. Здесь – сквозные технические сценарии двух ключевых процессов на уровне взаимодействия фронтенда, бэкенда и базы данных; диаграммы последовательностей пяти ключевых технических сценариев, детализирующих часть процессов из подраздела 1.2, приведены в приложении А.

5.2 Создание записи (бронирование)

Технический сценарий создания записи на уровне взаимодействия фронтенда, бэкенда и базы данных показан в соответствии с рисунком 6.

Повторная проверка доступности слота на шаге 6 на уровне СУБД, а не только на уровне кода перед вставкой, – прямая реализация цели «0 инцидентов двойного бронирования» из технического задания (подраздел 2.2): при конкурентной попытке два запроса могут одновременно пройти проверку на шаге 1–4, но лишь один пройдёт INSERT на шаге 6, второй получит 409 от самой базы данных.

5.3 Первый запуск системы (мастер /setup)

Сценарий первого запуска, выполняемый мастером настройки /setup, представлен в соответствии с рисунком 7.

Маршрут /setup проверяется единым глобальным `router.beforeEach`-хуком фронтенда перед *каждой* навигацией (раздел 2), поэтому попасть в остальную часть приложения, минуя этот сценарий, нельзя; после успешного шага 6 статус настройки самоблокируется навсегда (руководство администратора, подраздел 3.4).

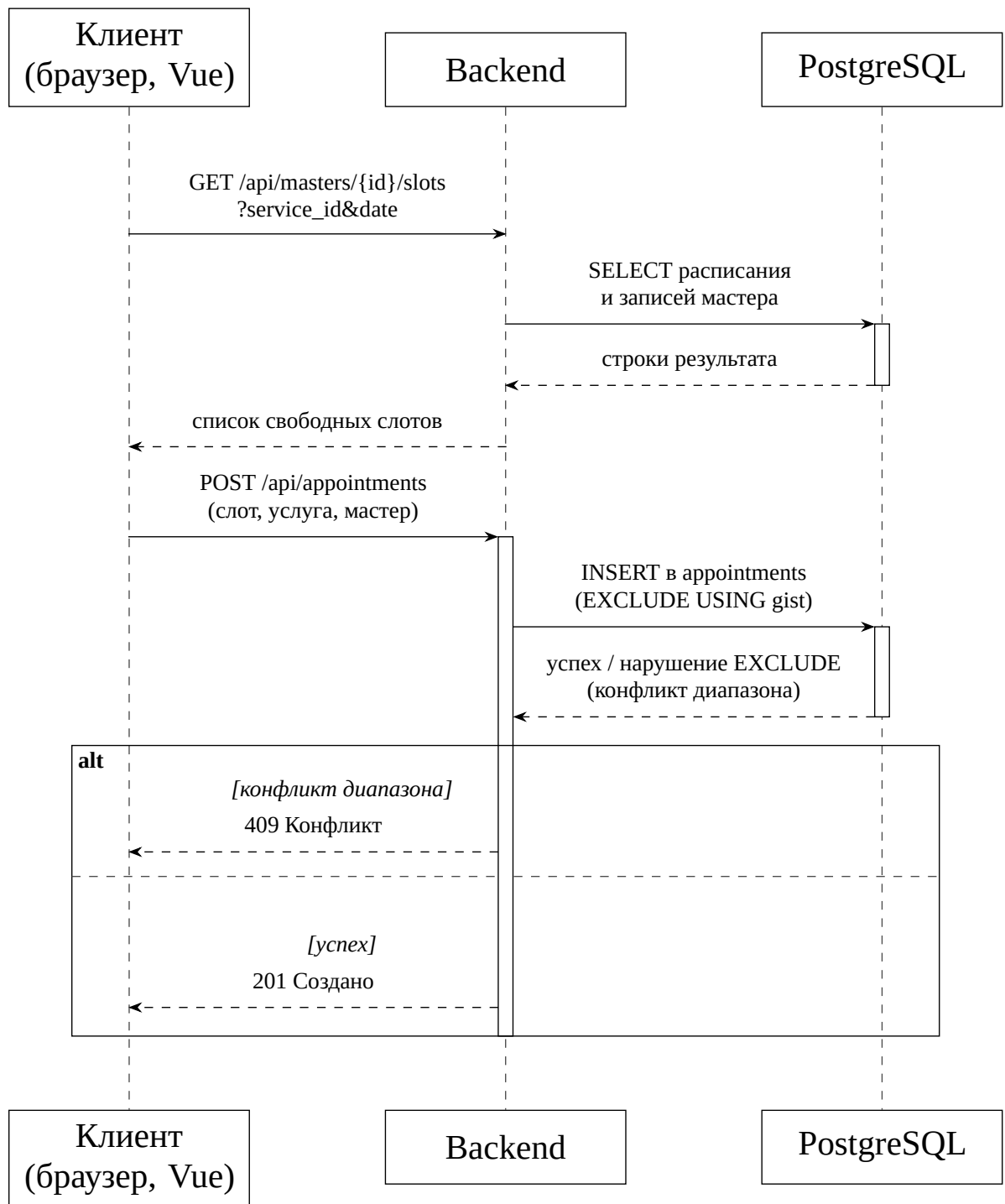


Рисунок 6 – Диаграмма последовательности: создание записи

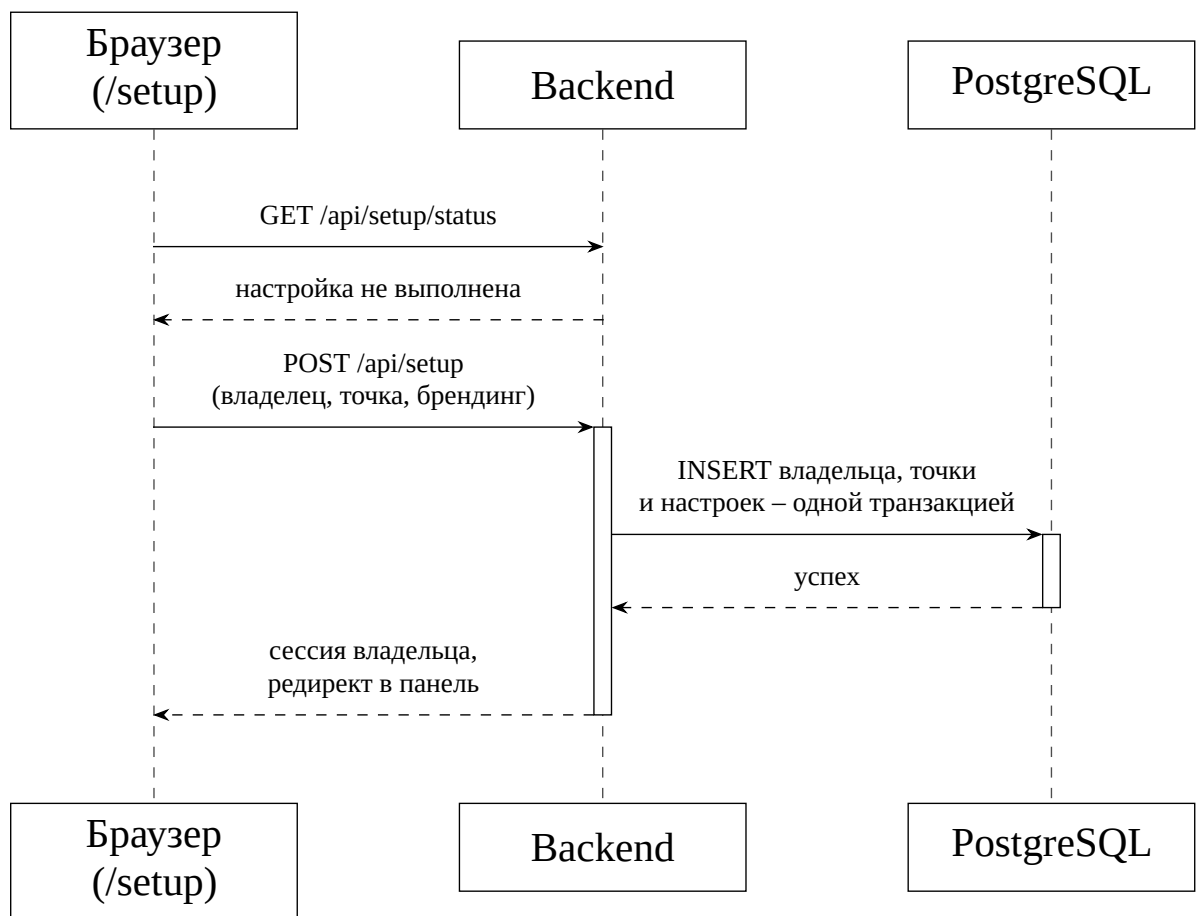


Рисунок 7 – Диаграмма последовательности: первый запуск (/setup)

5.4 Подключение аналитики

В системе три независимых источника аналитики, которые не стоит путать между собой:

- **Веб-аналитика (Яндекс.Метрика) – подключена и активна.** Счётчик загружается отдельным статическим файлом (/metrika.js), а не инлайн-скриптом в index.html, – ради политики безопасности контента (CSP): script-src остаётся 'self' без исключения 'unsafe-inline'. Использование раскрыто в §8 политики конфиденциальности (2.4).
- **Мониторинг ошибок (Sentry) – код готов, отключён по умолчанию.** Инициализация присутствует и на бэкенде, и на фронтенде, но выполняется только при заданном SENTRY_DSN/VITE_SENTRY_DSN.
- **Бизнес-аналитика – собственные отчёты по данным приложения.** Раздел «Статистика» (KPI-плитки, график регистраций за 30 дней) и раздел «Отчёты» (выручка и загруженность по дням/мастерам/услугам с экспортом в Excel) – оба на данных из собственной БД проекта, без внешнего сервиса. Реализуют цель технического задания «формирование отчёта по выручке и загруженности без привлечения разработчика» (подраздел 2.2); полное описание экранов – руководство пользователя, пункты 4.3.1 и 4.3.2.

5.5 Размещение в сети Интернет

Полный порядок развёртывания, включая перечень необходимых портов, подключение домена и TLS, первичную настройку и восстановление после сбоя, – предмет отдельного документа (руководство администратора, разделы 1–4) и здесь не повторяется. Кратко: система разворачивается как согласованный набор Docker-контейнеров [12] (база данных, backend, frontend, реверс-прокси Caddy [13] с автоматическим TLS-сертификатом Let's Encrypt, ежедневное резервное копирование), наружу из этой топологии открыты только порты 80/443 контейнера Caddy, обновление выполняется вручную одной командой (scripts/deploy.sh) без автоматического деплоя по CI (3.4).

ЗАКЛЮЧЕНИЕ

В ходе разработки создана и доведена до готовности к развёртыванию CMS «Сайтама» – веб-приложение для автоматизации онлайн-записи и операционной деятельности сети барбершопов.

Соответствие целям технического задания. Ключевые измеримые цели (подраздел 2.2 технического задания) обеспечены технически: двойное бронирование исключено ограничением целостности СУБД, а не организационно (раздел 5); индивидуальный прайс по мастеру, единое хранилище данных и отчёт по выручке/загруженности без участия разработчика – реализованы и доступны в интерфейсе администратора (5.4). Часть целей (снижение доли неявок за счёт напоминаний, доля клиентов, выбирающих вход через VK ID) сформулирована как показатель, измеримый только на реальной эксплуатации.

Направления дальнейшего развития. Второй OAuth-провайдер и дополнительные каналы уведомлений можно добавить без изменения архитектуры (раздел 2 уже отделяет бизнес-логику сервисов от конкретного канала доставки/провайдера); горизонтальное масштабирование backend потребует пересмотра внутрипроцессных механизмов защиты от перебора паролей и ограничения частоты запросов (раздел 3) в пользу общего хранилища состояния; полноценный журнал аудита – отдельная таблица с записью каждой критичной операции, а не только попыток входа.

Итоговое состояние проекта, полный перечень функциональных требований и инструкции по эксплуатации – см. соответственно техническое задание, руководство пользователя и руководство администратора; весь комплект документации оформлен с учётом общих требований к программным документам [14].

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

- 1) FastAPI – официальная документация [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 29.08.2026).
- 2) SQLAlchemy 2.0 – официальная документация [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 29.08.2026).
- 3) PostgreSQL 16 – официальная документация [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 29.08.2026).
- 4) ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М.: Стандартинформ, 2020.
- 5) ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – М.: Изд-во стандартов, 1978.
- 6) Vue.js – официальная документация [Электронный ресурс]. – URL: <https://vuejs.org/> (дата обращения: 29.08.2026).
- 7) Pydantic – официальная документация [Электронный ресурс]. – URL: <https://docs.pydantic.dev/> (дата обращения: 29.08.2026).
- 8) Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных».
- 9) Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации».
- 10) ГОСТ Р ИСО/МЭК 15408-3-2008. Информационная технология. Методы и средства обеспечения безопасности. Критерии оценки безопасности информационных технологий. Часть 3. – М.: Стандартинформ, 2008.
- 11) Alembic – официальная документация [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 29.08.2026).
- 12) Docker Compose – официальная документация [Электронный ресурс]. – URL: <https://docs.docker.com/compose/> (дата обращения: 29.08.2026).
- 13) Caddy – официальная документация [Электронный ресурс]. –

URL: <https://caddyserver.com/docs/> (дата обращения: 29.08.2026).

14) ГОСТ 19.105-78. Единая система программной документации. Общие требования к программным документам. – М.: Изд-во стандартов, 1978.

ПРИЛОЖЕНИЕ А

Диаграммы последовательностей

Рисунки А.1–А.5 иллюстрируют порядок взаимодействия участников для ключевых бизнес-процессов системы, детализированных в карточках функциональных требований технического задания (подраздел 4.2).

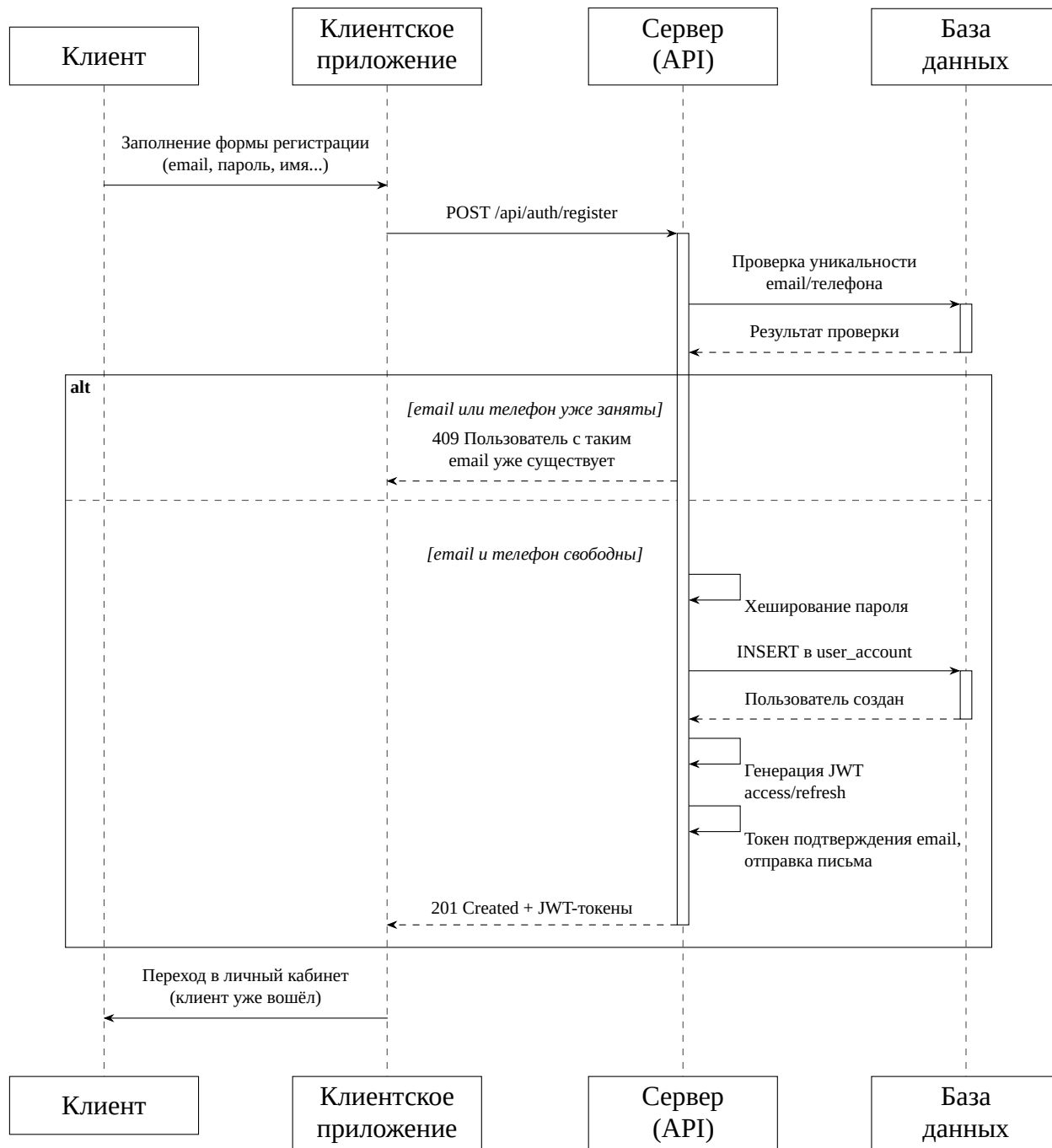


Рисунок А.1 – Диаграмма последовательностей бизнес-процесса «Регистрация» (ФТ-001)

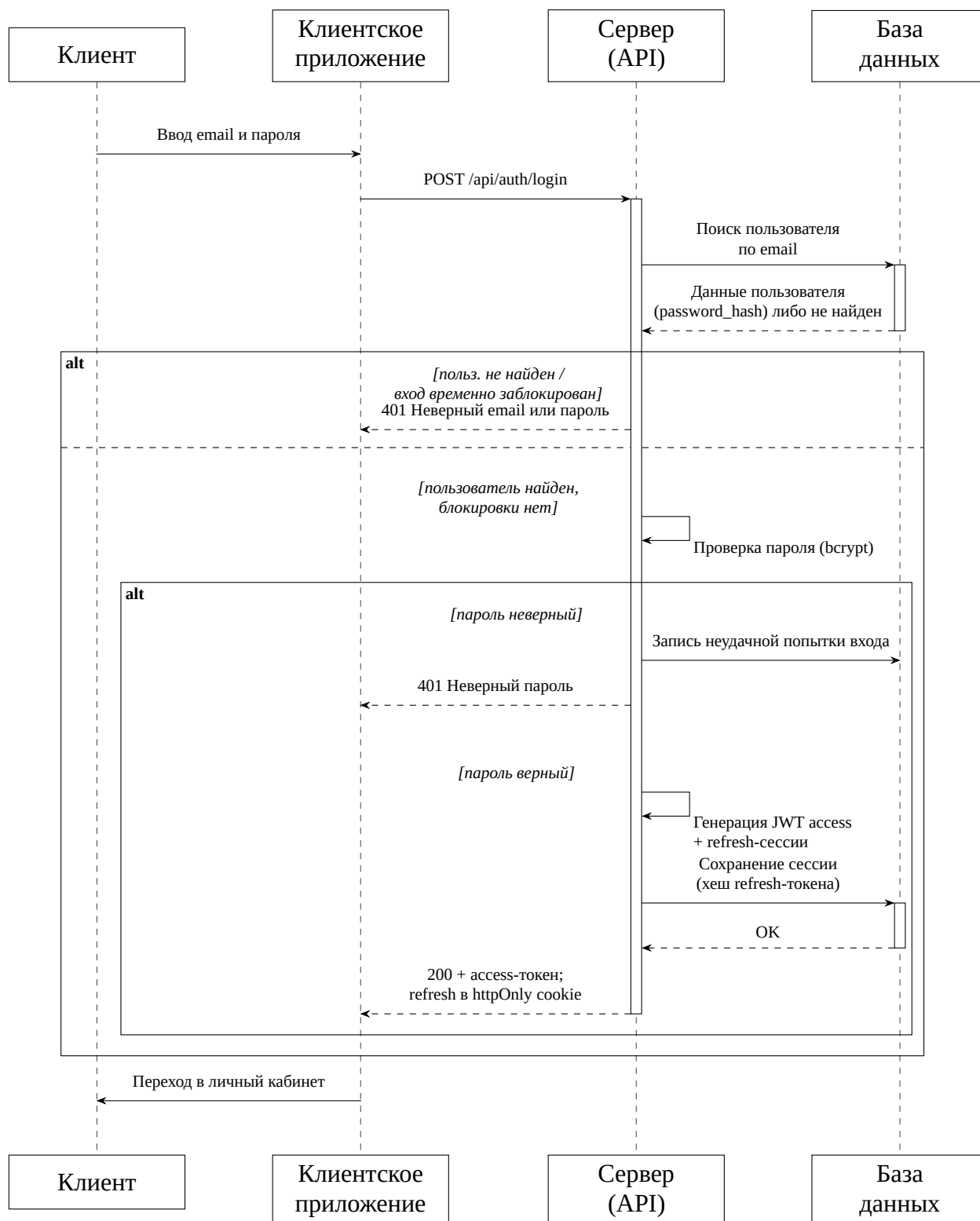


Рисунок А.2 – Диаграмма последовательностей бизнес-процесса «Вход по email и паролю»

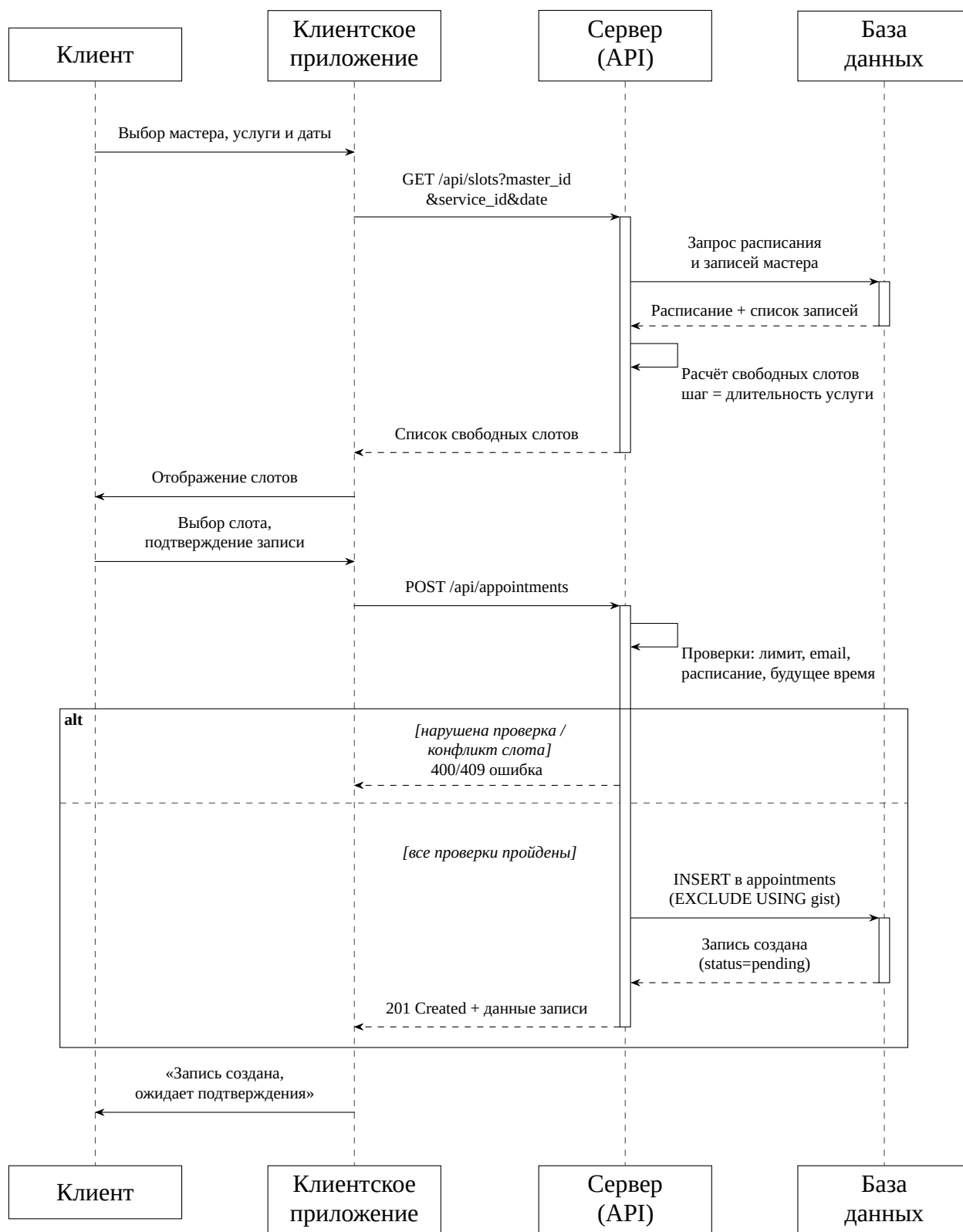


Рисунок А.3 – Диаграмма последовательностей бизнес-процесса «Онлайн-запись на приём» (ФТ-004, ФТ-005)

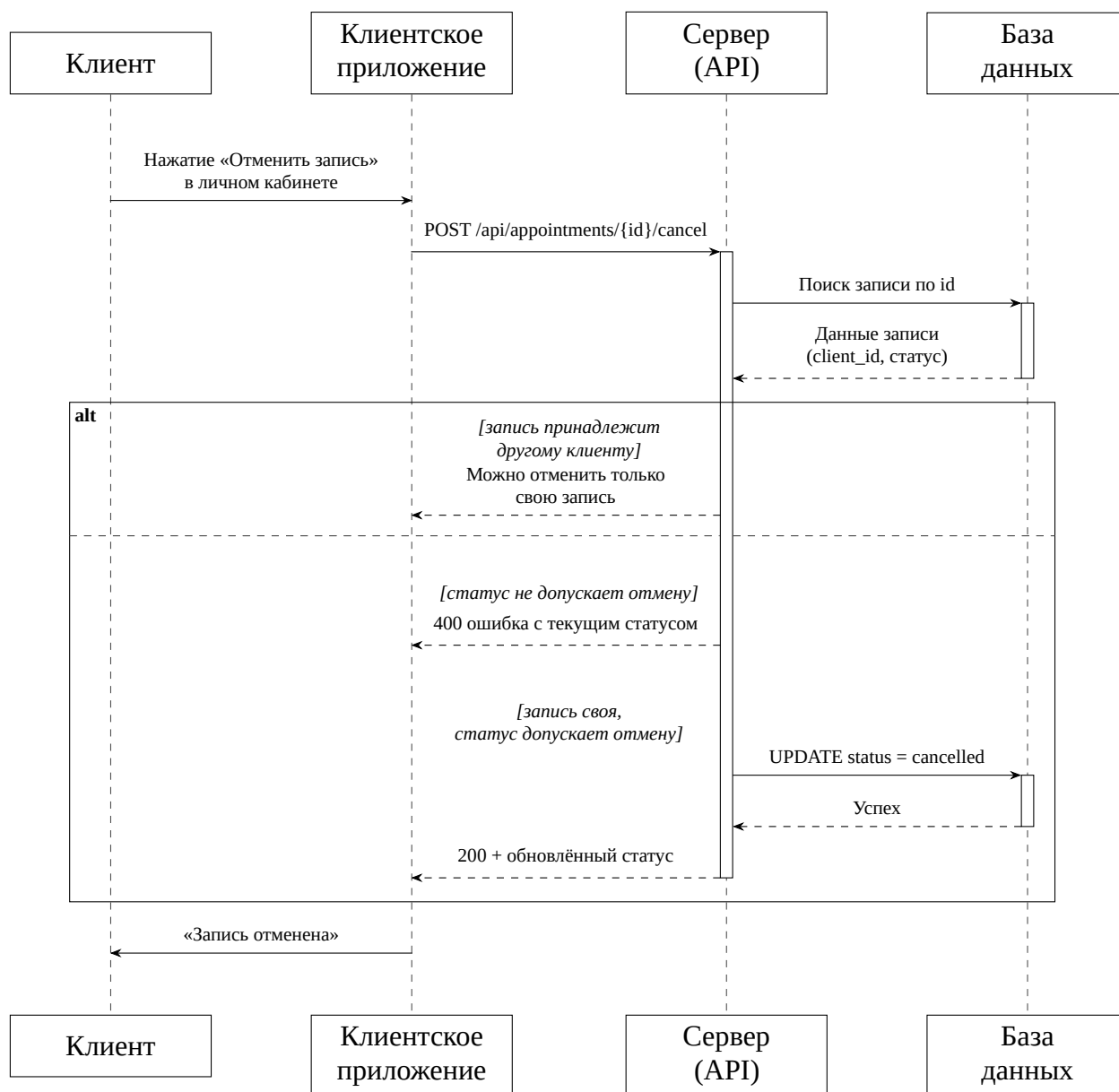


Рисунок А.4 – Диаграмма последовательностей бизнес-процесса «Отмена записи клиентом» (ФТ-006)

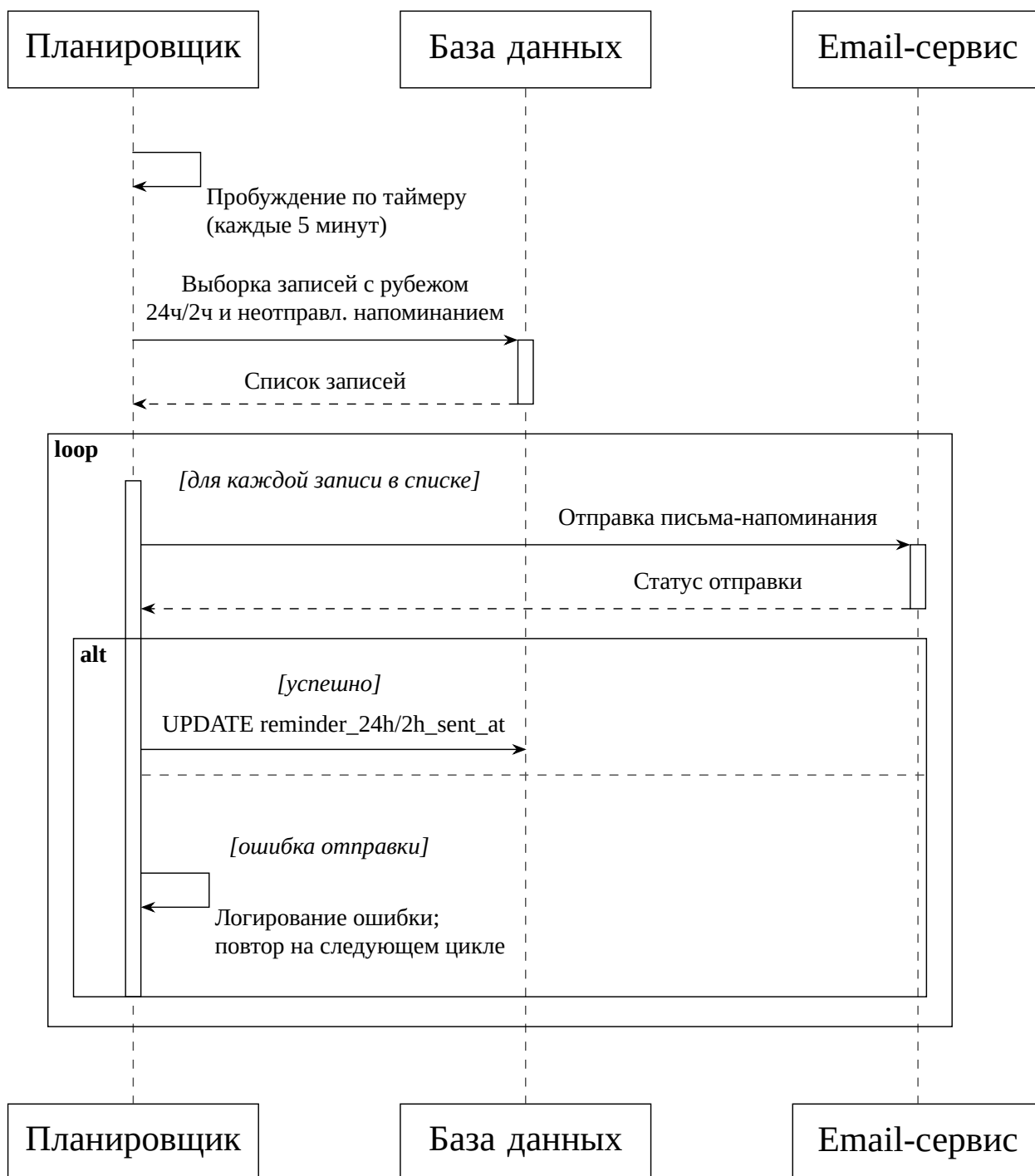


Рисунок А.5 – Диаграмма последовательностей бизнес-процесса
«Автоматическая отправка напоминаний» (ФТ-008)